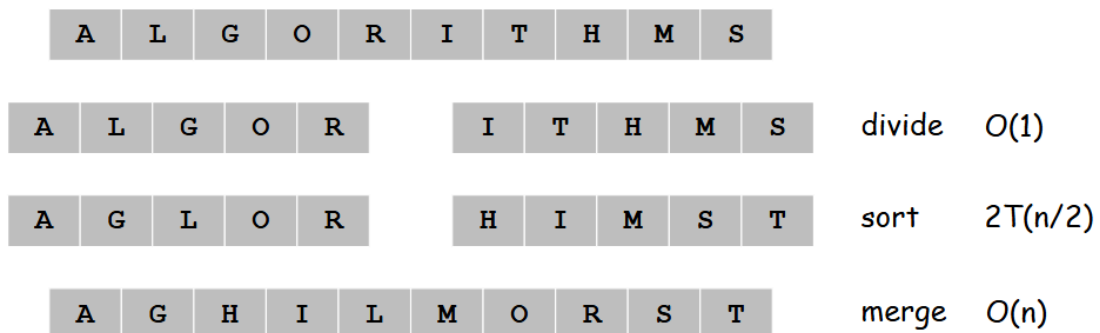


Divide And Conquer

[05divide-and-conquer.pdf](#)

分治思想，分而治之

归并排序（数据结构大记忆恢复）



其实理解分治思想，还是要先把递归想清楚，说到递归，递归最重要的思想就是“拆分 + 回溯”，所以想到分治思想，我们脑子里要浮现出两个画面：

- ①怎么拆下去的
- ②怎么拼回来的。

（真的想要形象化理解的话，推荐抖音随便搜一个视频，笨人在熬夜刷dy的时候刷到了归并算法的可视化，就这样在数据结构期末考完三个月后才明白。）

数组合并算法：

假设有：

$$A = [2, 5, 7, _, _, _], \quad m = 3$$

$$B = [1, 3, 6], \quad n = 3$$

如果我们从前往后合并，那么一旦你想把 1 放进 $A[0]$ ，原来的 2 就被挤掉了，后面还得想办法保存，很麻烦。但如果从后往前：

- 先看两边最后一个有效元素：7 和 6
- 把更大的 7 放到最后

- 再比较 5 和 6
- 把 6 放到倒数第二个位置

这样完全不会破坏前面还没处理的数据。这就是“原地合并”。估计这个不会考，我们就记住，**合并数组的时间复杂度一般是 $O(n)$ 就好。**

利用递推式分析归并排序的次数：

归并排序的思想是：

1. 把长度为 n 的数组分成两半
2. 递归地排好左半边
3. 递归地排好右半边
4. 把两个有序子数组合并

所以，处理一个规模为 n 的问题，代价由三部分组成：

- 排左半边： $T(\lceil n/2 \rceil)$
- 排右半边： $T(\lfloor n/2 \rfloor)$
- 最后合并：线性代价，大约是 n

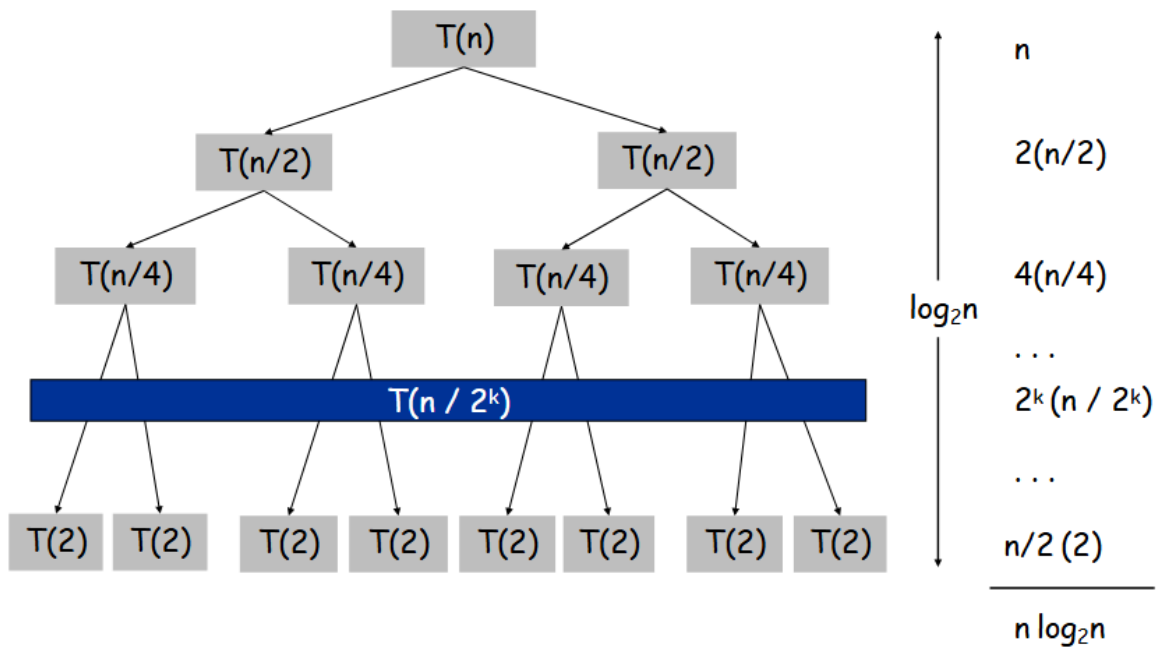
于是得到递推式：

$$T(n) \leq \begin{cases} 0, & n = 1 \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n, & n > 1 \end{cases}$$

一般情况下，也就是每次二分时，我们可以写出更简洁的递推式：

$$T(n) = \begin{cases} 0, & n = 1 \\ 2T(n/2) + n, & n > 1 \end{cases}$$

也就是 总成本 = 递归成本 + 本层合并成本，并画出形如这样的递归树：



根据这样的分法，一共可分 $\log n$ 层，而每一层都要合并 n 个元素，所以一共就是 $O(n \log n)$ 。这是利用递归树的理解方法，接下来还有一个方法涉及到一些数学上的小技巧思：

我们从递推式出发：

$$T(n) = 2T(n/2) + n$$

两边同时除以 n ：

$$\frac{T(n)}{n} = \frac{2T(n/2)}{n} + 1$$

因为

$$\frac{2T(n/2)}{n} = \frac{T(n/2)}{n/2}$$

$$\text{所以: } \frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1$$

继续往下代：

$$\frac{T(n/2)}{n/2} = \frac{T(n/4)}{n/4} + 1$$

代回去：

$$\frac{T(n)}{n} = \frac{T(n/4)}{n/4} + 1 + 1$$

再继续：

$$\frac{T(n)}{n} = \frac{T(n/8)}{n/8} + 1 + 1 + 1$$

.....

一直展开到规模降到 1：

$$\frac{T(n)}{n} = \frac{T(1)}{1} + \underbrace{1 + 1 + \dots + 1}_{\log_2 n \uparrow}$$

由于 $T(1) = 0$ ，所以：

$$\frac{T(n)}{n} = \log_2 n$$

$$T(n) = n \log_2 n$$

反正我自己想是想不出来，总之能看懂就是了.....它的妙处在于：

通过除以当前规模，把“每一层贡献一个 n ”转化成了“每递归一层就加 1”。

数归法证明：

- Base case: $n = 1$.
- Inductive hypothesis: $T(n) = n \log_2 n$.
- Goal: show that $T(2n) = 2n \log_2 (2n)$.

$$\begin{aligned}
 T(2n) &= 2T(n) + 2n \\
 &= 2n \log_2 n + 2n \\
 &= 2n(\log_2(2n) - 1) + 2n \\
 &= 2n \log_2(2n)
 \end{aligned}$$

这些天才们都是怎么想到这个方法的.....

一般情形下的递推式：

我们研究的是：

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

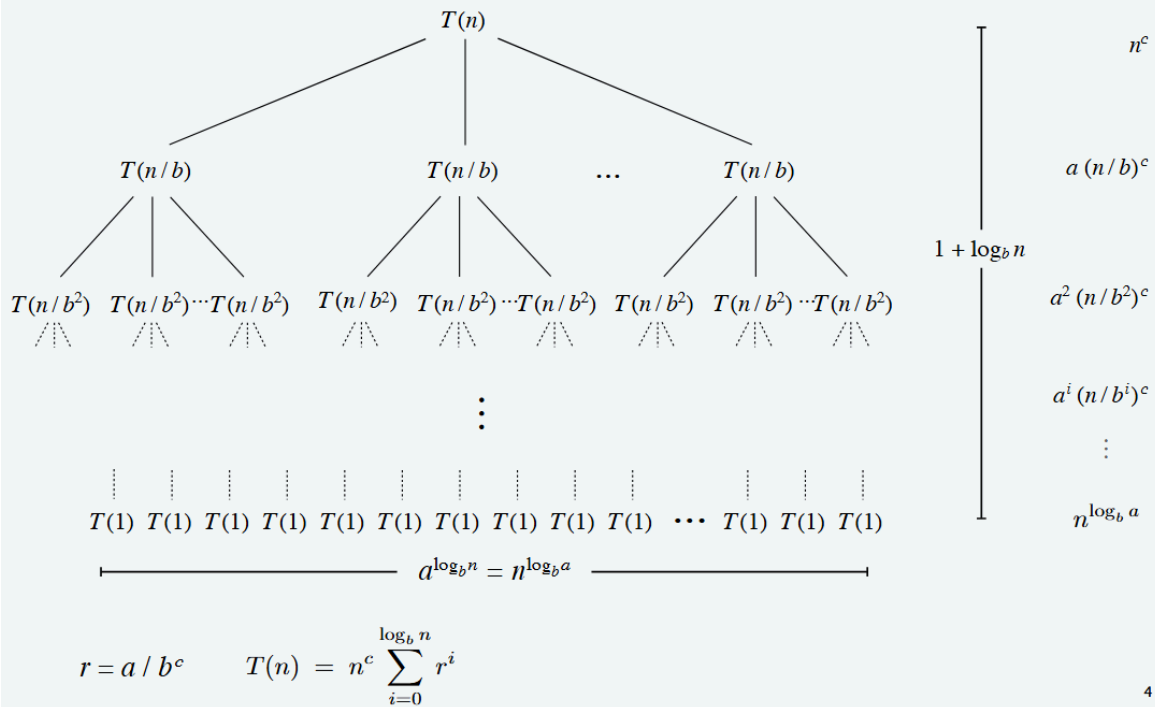
其中通常默认：

- $a \geq 1$: 子问题个数
- $b \geq 2$: 每个子问题的规模缩小为原来的 $1/b$
- $f(n) \geq 0$: 这一层“分解 + 合并 + 额外处理”的代价

注意：对于一些符号我们要明白是什么意思， a 是有效子问题的个数，不等价于一次分为多少份，比如说归并排序每次分为两份，子问题数量也为两份，但二分查找，每次分为两份，但是只找一半，所以子问题数量 $a = 1$ 。

同样，**递归树**还是我们理解问题的关键：

Suppose $T(n)$ satisfies $T(n) = a T(n/b) + n^c$ with $T(1) = 1$, for n a power of b .



式子本质上就是在说：

一个规模为 n 的问题，分裂成 a 个规模为 n/b 的子问题，当前层额外再花 $f(n)$ 的成本。

所以最自然的图像就是一棵树。我们来拆解一下这个**递归树的各项信息**：首先，第 0 层：根节点只有 1 个问题，规模是 n ，代价是：

$$f(n)$$

第 1 层有 a 个子问题，每个规模是 n/b ，所以这一层总代价是：

$$a \cdot f(n/b)$$

第 2 层，每个子问题再分成 a 个，所以总共有 a^2 个节点，每个规模是 n/b^2 ，所以这一层总代价是：

$$a^2 f(n/b^2)$$

接下来，一般情况下，当我们来到第 i 层时：

- 节点数: a^i
- 每个子问题规模: n/b^i
- 这一层总代价:

$$a^i f(n/b^i)$$

同时, 由于每往下一层, 就分为 b 份, 一直这样直到规模降为 1:

$$\frac{n}{b^h} = 1$$

$$h = \log_b n$$

所以整棵递归树大概有 $\log_b n$ 层。

当 $f(n) = n^c$ 时的研究:

$f(n)$ 代表的是每一层处理都是多项式级别时间复杂度, 由于多项式级别时间是我们着重研究的, 所以我们先看:

$$T(n) = aT(n/b) + n^c$$

这里 n^c 表示当前层的代价是一个多项式量级。对于:

$$T(n) = aT(n/b) + n^c$$

第 i 层总代价为:

$$a^i \left(\frac{n}{b^i} \right)^c$$

$$a^i \cdot \frac{n^c}{b^{ic}} = n^c \left(\frac{a}{b^c} \right)^i$$

$$\text{设 } r = \frac{a}{b^c}$$

那么第 i 层总代价就是：

$$n^c r^i$$

$$\text{故有： } T(n) \approx n^c \sum_{i=0}^{\log_b n} r^i$$

分析这个递推，最后变成分析一个等比数列。

于是我们就可以分类讨论了：

$$T(n) = n^c \sum_{i=0}^{\log_b n} r^i = \begin{cases} \Theta(n^c) & \text{if } r < 1 & c > \log_b a & \longleftarrow \text{cost dominated by cost of root} \\ \Theta(n^c \log n) & \text{if } r = 1 & c = \log_b a & \longleftarrow \text{cost evenly distributed in tree} \\ \Theta(n^{\log_b a}) & \text{if } r > 1 & c < \log_b a & \longleftarrow \text{cost dominated by cost of leaves} \end{cases}$$

因为等比数列的行为，取决于公比 r ：

- $r < 1$ ：越往下越小（成本集中在树顶，所以主要就以第一层，一次 $f(n)$ 为主）
- $r = 1$ ：每层一样大（成本均匀分布，所以这个最好记，就是 层数*每层的代价）
- $r > 1$ ：越往下越大（成本集中在树底， $a^{\log_b n} = n^{\log_b a}$ 其实是叶子层结点的数量，集中在树底时，就看叶子数）

r 的直观意义就是“层成本增长率”

- a ：这一层往下会分出多少个子问题
- b^c ：每个子问题因为规模缩小，单个节点成本会缩小多少倍

于是：

$$r = \frac{\text{节点数增长倍数}}{\text{单个节点成本缩小倍数}}$$

“子问题个数变多”和“单个子问题变轻松”这两种力量在打架， r 就是博弈的结果。

